

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	K Tejaswi Reddy	Reg. No.:	192371036
Section / Batch:	CS-BIOSCI	Date:	29-08-2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions	Marks
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – CONSTRAINT BASED PROBLEM SOLVING

CS4001 DAA - SED D - Class

SIMATS Online Lab

Ask Gemini

Net secure

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1524

Search

Share

Star

Download

School

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

KOMMURU TEJASWI REDDY

REGISTRATION

CO5_AT1_Constraint-Based Problem Solving

Back

Language: **python** C++ Java

QUESTIONS

Q1
Backtracking and Tractability
10 marks

Q2
Backtracking and Tractability
10 marks

2/2 answered

BackTracking and Tractability

10 marks

Longest Path Problem (NP-Hard Problem)

Find the longest simple path between two vertices in a graph. Clearly define constraints such as vertex non-repetition and path continuity. Analyze how constraints affect solution feasibility. Develop a backtracking-based approach to explore paths. Apply pruning to eliminate shorter paths. Justify correctness and explain NP-Hard classification.

Your Code

```
1 class LongestPathSolver:
2     def __init__(self, graph, max_vertices):
3         self.graph = graph
4         self.visited = [False] * max_vertices
5         self.path = []
6         self.best_path = []
7         self.max_length = 0
8
9     def is_safe(self, vertex):
10         return not self.visited[vertex]
11
12     def backtrack(self, current, destination, current_length):
13         self.visited[current] = True
14         self.path.append(current)
15         if current == destination:
16             if current_length > self.max_length:
17                 self.max_length = current_length
```

Input

Enter Input

Output

Longest simple path from 0 to 4:
0, 1, 2, 3, 4
Path length (number of edges): 4

CS4001 DAA - SED D - Class

SIMATS Online Lab

Ask Gemini

Net secure

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1524

Search

Share

Star

Download

School

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

KOMMURU TEJASWI REDDY

REGISTRATION

CO5_AT1_Constraint-Based Problem Solving

Back

Language: **python** C++ Java

QUESTIONS

Q1
Backtracking and Tractability
10 marks

Q2
Backtracking and Tractability
10 marks

2/2 answered

BackTracking and Tractability

10 marks

Longest Path Problem (NP-Hard Problem)

Find the longest simple path between two vertices in a graph. Clearly define constraints such as vertex non-repetition and path continuity. Analyze how constraints affect solution feasibility. Develop a backtracking-based approach to explore paths. Apply pruning to eliminate shorter paths. Justify correctness and explain NP-Hard classification.

Your Code

```
1 class LongestPathSolver:
2     def __init__(self, graph, max_vertices):
3         self.graph = graph
4         self.visited = [False] * max_vertices
5         self.path = []
6         self.best_path = []
7         self.max_length = 0
8
9     def is_safe(self, vertex):
10         return not self.visited[vertex]
11
12     def backtrack(self, current, destination, current_length):
13         self.visited[current] = True
14         self.path.append(current)
15         if current == destination:
16             if current_length > self.max_length:
17                 self.max_length = current_length
18                 self.best_path = self.path.copy()
19         elif:
20             for neighbor in self.graph.get(current, []):
21                 if self.is_safe(neighbor):
22                     self.backtrack(neighbor, destination, current_length + 1)
23
24         self.visited[current] = False
25         self.path.pop()
26
27     def find_longest_path(self, source, destination):
28         self.backtrack(source, destination, 0)
29         return self.best_path, self.max_length
30
31 if __name__ == "__main__":
32     graph = {
33         0: [1, 2],
34         1: [0, 2, 3],
35         2: [0, 1, 3],
36         3: [1, 2, 4],
37         4: [3]
38     }
39
40     max_vertices = 5
41     source, destination = 0, 4
42
43     solver = LongestPathSolver(graph, max_vertices)
44     path, length = solver.find_longest_path(source, destination)
45
46     print(f"Longest simple path from {source} to {destination}: {path}")
47     print(f"Path length (number of edges): {length}")
```

Input

Enter Input

Output

Longest simple path from 0 to 4:
0, 1, 2, 3, 4
Path length (number of edges): 4

CUADUT DAA - SECT D - Class

SIMATS Online Lab

Ask Gemini

Net secure simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1524

SIMATS
ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KAMMARU TEJASWI REDDY

REGISTRATION

QUESTIONS

Q1
Backtracking and Tractability
30 marks

Q2
Backtracking and Tractability
30 marks

0/0 answered

BackTracking and Tractability

30 marks

Independent Set Problem (NP-Complete / Backtracking Problem)
Given a graph, find the largest set of vertices with no edges between them. Clearly define constraints related to vertex independence. Analyze how constraints affect feasible subsets. Develop a backtracking-based solution. Apply pruning techniques to discard invalid sets. Justify how your solution works and explain NP-Complete classification

Your Code

```
def is_independent(vertices, current_set, graph):
    for v in current_set:
        if v in graph[current_set]:
            return False
    return True

def backtrack(graph, vertices, index, current_set, best_set):
    if len(current_set) > len(best_set):
        best_set = current_set
        return

    if index == len(vertices):
        return

    if len(current_set) > len(best_set):
        best_set = current_set
        return

    vertex = vertices[index]
    if is_independent(vertices, current_set, graph):
        current_set.append(vertex)
        backtrack(graph, vertices, index + 1, current_set, best_set)
        current_set.pop()
    backtrack(graph, vertices, index + 1, current_set, best_set)
```

Input

vertex graph

Output

Maximum Independent Set: [1, 3]
Size: 2

CUADUT DAA - SECT D - Class

SIMATS Online Lab

Ask Gemini

Net secure simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1524

SIMATS
ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

KAMMARU TEJASWI REDDY

REGISTRATION

Backtracking and Tractability

30 marks

Independent Set Problem (NP-Complete / Backtracking Problem)
Given a graph, find the largest set of vertices with no edges between them. Clearly define constraints related to vertex independence. Analyze how constraints affect feasible subsets. Develop a backtracking-based solution. Apply pruning techniques to discard invalid sets. Justify how your solution works and explain NP-Complete classification

Your Code

```
def is_independent(vertices, current_set, graph):
    for v in current_set:
        if v in graph[current_set]:
            return False
    return True

def backtrack(graph, vertices, index, current_set, best_set):
    if len(current_set) > len(best_set):
        best_set = current_set
        return

    if index == len(vertices):
        return

    if len(current_set) > len(best_set):
        best_set = current_set
        return

    vertex = vertices[index]
    if is_independent(vertices, current_set, graph):
        current_set.append(vertex)
        backtrack(graph, vertices, index + 1, current_set, best_set)
        current_set.pop()
    backtrack(graph, vertices, index + 1, current_set, best_set)
```

Input

vertex graph

Output

Maximum Independent Set: [1, 3]
Size: 2

Code of Conduct

I Tejaswi Reddy K (192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Tejaswi Reddy K

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Question No.	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____